

Τμήμα Μηχ. Η/Υ & Πληροφορικής
Πολυτεχνική Σχολή, Πανεπιστήμιο Ιωαννίνων
ΜΥΥ601 Λειτουργικά Συστήματα

Υλοποίηση πολυνηματικής λειτουργίας σε μηχανή αποθήκευσης δεδομένων.

ΕΡΓΑΣΙΑ ΤΩΝ ΦΟΙΤΗΤΩΝ:

ΕΥΑΓΓΕΛΟΣ ΜΠΑΣΔΑΒΑΝΟΣ ΑΜ:4962

ΚΩΝΣΤΑΝΤΙΝΟΣ ΚΑΨΗΜΑΛΗΣ ΑΜ:4698

ΡΩΜΑΝΟΣ ΚΩΤΣΗΣ ΑΜ:4714

ΠΕΡΙΕΧΟΜΕΝΑ

• 1 ^ο ερώτημα – Πειραματικές Μετρήσεις	<u>2</u>
• Απλή Προσέγγιση	<u>6</u>
• Υλοποίηση Λειτουργίας ReadWrite	<u>8</u>
• Πειραματικές Μετρήσεις Απλής Προσέγγισης	<u>20</u>
• Πειραματικές Μετρήσεις Προηγμένης Προσέγγισης	<u>26</u>
• Άλλες Πληροφορίες	<u>30</u>

1^ο Ερώτημα

Στην αρχική φάση της εργασίας διερευνήσαμε τα υπάρχοντα νήματα που υπάρχουν στην εφαρμογή και τη μηχανή καθώς και τις λειτουργίες συγχρονισμού που υπήρχαν ήδη στην υλοποίηση που μας δόθηκε. Κατανοήσαμε την λειτουργία των βασικών μεθόδων και την μεταξύ τους συνεργασία ώστε να επιτευχθεί η επιτυχής λειτουργία της μηχανής αποθήκευσης δεδομένων πριν αρχίσουμε να την τροποποιούμε. Με σκοπό να κατανοήσουμε πλήρως την λειτουργία της μηχανής, κάναμε πειραματικές μετρήσεις για διάφορες τιμές των κλειδιών. Για τιμές κλειδιών μικρότερες των 100.000 το κόστος ήταν ίσο με 0 sec οπότε θεωρήσαμε άσκοπο να τα συμπεριλάβουμε στις μετρήσεις. Παρακάτω, παρατίθενται στιγμιότυπα οθόνης από τις μετρήσεις. Για να λάβουμε τα αποτελέσματα των εγγραφών, χρησιμοποιήσαμε την εντολή `./kiwi-bench write x`, όπου `x` ο αριθμός των κλειδιών για τον οποίο θέλουμε να κάνει την μέτρηση κάθε φορά. Αντίστοιχα, για να παρατηρήσουμε τα κόστη των αναγνώσεων χρησιμοποιήσαμε την εντολή `./kiwi-bench read x`, όπου `x` ο αριθμός των κλειδιών για τον οποίο θέλουμε να κάνει την μέτρηση κάθε φορά.

Κόστη εγγραφών εκατό χιλιάδων ($x = 100.000$) κλειδιών:

```
+-----+
|Random-Write  (done:100000): 0.000010 sec/op; 100000.0 writes/sec(estimated); cost:1.000(sec);
```

Εικόνα 1

```
+-----+
|Random-Write  (done:100000): 0.000020 sec/op; 50000.0 writes/sec(estimated); cost:2.000(sec);
```

Εικόνα 2

Τρέξαμε το πρόγραμμα 10 φορές για 100.000 εγγραφές και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 100.000 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 1 δευτερόλεπτο (εικόνα 1) ή η ρυθμαπόδοση ίση με 50.000 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 2 δευτερόλεπτα (εικόνα 2).

Κόστη αναγνώσεων εκατό χιλιάδων ($x = 100.000$) κλειδιών:

```
+-----+
|Random-Read   (done:100000, found:100000): 0.000010 sec/op; 100000.0 reads /sec(estimated); cost:1.000(sec)
```

Εικόνα 3

```
+-----+
|Random-Read   (done:100000, found:100000): 0.000020 sec/op; 50000.0 reads /sec(estimated); cost:2.000(sec)
```

Εικόνα 4

Τρέξαμε το πρόγραμμα 10 φορές για 100.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 100.000 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 1 δευτερόλεπτο (εικόνα 3) ή η ρυθμαπόδοση ίση με 50.000 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 2 δευτερόλεπτα (εικόνα 4). Έπειτα, εκτελέσαμε μέτρηση για 200.000 εγγραφές.

Κόστη εγγραφών διακοσίων χιλιάδων ($x = 200.000$) κλειδιών:

```
+-----+
|Random-Write  (done:200000): 0.000010 sec/op; 100000.0 writes/sec(estimated); cost:2.000(sec);
+-----+
```

Εικόνα 5

```
+-----+
|Random-Write  (done:200000): 0.000015 sec/op; 66666.7 writes/sec(estimated); cost:3.000(sec);
+-----+
```

Εικόνα 6

Τρέξαμε το πρόγραμμα 10 φορές για 200.000 εγγραφές και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 100.000 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 2 δευτερόλεπτα (εικόνα 5) ή η ρυθμαπόδοση ίση με 66.666,7 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 3 δευτερόλεπτα (εικόνα 6).

Κόστη αναγνώσεων διακοσίων χιλιάδων ($x = 200.000$) κλειδιών:

```
+-----+
|Random-Read   (done:200000, found:200000): 0.000010 sec/op; 100000.0 reads /sec(estimated); cost:2.000(sec)
+-----+
```

Εικόνα 7

```
+-----+
|Random-Read   (done:200000, found:200000): 0.000015 sec/op; 66666.7 reads /sec(estimated); cost:3.000(sec)
+-----+
```

Εικόνα 8

Τρέξαμε το πρόγραμμα 10 φορές για 200.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 100.000 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 2 δευτερόλεπτα (εικόνα 7) ή η ρυθμαπόδοση ίση με 66.666,7 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 3 δευτερόλεπτα (εικόνα 8). Έπειτα, εκτελέσαμε μέτρηση για 300.000 εγγραφές.

Κόστη εγγραφών τριακοσίων χιλιάδων ($x = 300.000$) κλειδιών:

```
+-----+
|Random-Write  (done:300000): 0.000013 sec/op; 75000.0 writes/sec(estimated); cost:4.000(sec);
+-----+
```

Εικόνα 9

```
+-----+
|Random-Write  (done:300000): 0.000017 sec/op; 60000.0 writes/sec(estimated); cost:5.000(sec);
+-----+
```

Εικόνα 10

Τρέξαμε το πρόγραμμα 10 φορές για 300.000 εγγραφές και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 75.000 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 4 δευτερόλεπτα (εικόνα 9) ή η ρυθμαπόδοση ίση με 60.000 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 5 δευτερόλεπτα (εικόνα 10).

Κόστη αναγνώσεων τριακοσίων χιλιάδων ($x = 300.000$) κλειδιών:

```
+-----+
|Random-Read   (done:300000, found:300000): 0.000010 sec/op; 100000.0 reads /sec(estimated); cost:3.000(sec)
+-----+
```

Εικόνα 11

```
+-----+
|Random-Read  (done:300000, found:300000): 0.000017 sec/op; 60000.0 reads /sec(estimated); cost:5.000(sec)|
+-----+
```

Εικόνα 12

Τρέξαμε το πρόγραμμα 10 φορές για 300.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 100.000 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 3 δευτερόλεπτα (εικόνα 11) ή η ρυθμαπόδοση ίση με 60.000 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 5 δευτερόλεπτα (εικόνα 12). Έπειτα, εκτελέσαμε μέτρηση για 1.000.000 εγγραφές.

Κόστη εγγραφών ενός εκατομμυρίου ($x = 1.000.000$) κλειδιών:

```
+-----+
|Random-Write  (done:1000000): 0.000012 sec/op; 83333.3 writes/sec(estimated); cost:12.000(sec);
+-----+
```

Εικόνα 13

```
+-----+
|Random-Write  (done:1000000): 0.000014 sec/op; 71428.6 writes/sec(estimated); cost:14.000(sec);
+-----+
```

Εικόνα 14

Τρέξαμε το πρόγραμμα 10 φορές για 1.000.000 εγγραφές και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 83.333,3 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 12 δευτερόλεπτα (εικόνα 13) ή η ρυθμαπόδοση ίση με 71.428,6 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 14 δευτερόλεπτα (εικόνα 14).

Κόστη αναγνώσεων ενός εκατομμυρίου ($x = 1.000.000$) κλειδιών:

```
+-----+
|Random-Read  (done:1000000, found:1000000): 0.000008 sec/op; 125000.0 reads /sec(estimated); cost:8.000(sec)|
+-----+
```

Εικόνα 15

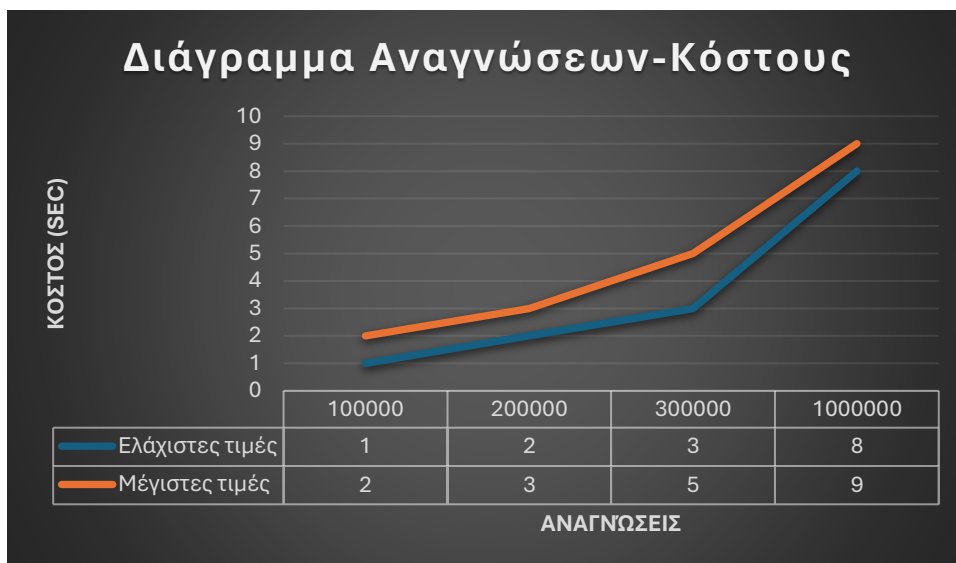
```
+-----+
|Random-Read  (done:1000000, found:1000000): 0.000009 sec/op; 111111.1 reads /sec(estimated); cost:9.000(sec)|
+-----+
```

Εικόνα 16

Τρέξαμε το πρόγραμμα 10 φορές για 1.000.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 125.000 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 8 δευτερόλεπτα (εικόνα 15) ή η ρυθμαπόδοση ίση με 111.111,1 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 9 δευτερόλεπτα (εικόνα 16).

Συμπερασματικά, παρατηρούμε ότι για ίδιες τιμές του x τα κόστη και η ρυθμαπόδοση μπορεί να διαφέρουν από πείραμα σε πείραμα, αφού σε κάθε πείραμα, οι συνθήκες (hardware) γύρω από τις οποίες λαμβάνει χώρα η κάθε μέτρηση, είναι διαφορετικές και επηρεάζουν την αποδοτικότητα του συστήματος. Συγκεκριμένα για τις εγγραφές, το νήμα που υλοποιεί την λειτουργία της συγχώνευσης επηρεάζει κάθε φορά το πόσο γρήγορα θα εκτελεστεί η λειτουργία με αποτέλεσμα μεγαλύτερα κόστη. Επιπλέον, είναι φανερό ότι για πολύ μεγάλο αριθμό κλειδιών (1.000.000) το κόστος για την λειτουργία της εγγραφής είναι αρκετά μεγαλύτερο από αυτό της ανάγνωσης, καθώς κατά την διάρκεια της λειτουργίας της εγγραφής, εκτελείται και το compaction/merge, το οποίο χρειάζεται επιπλέον χρόνο για να κάνει τις ανακατανομές στο memtable και στο sst.

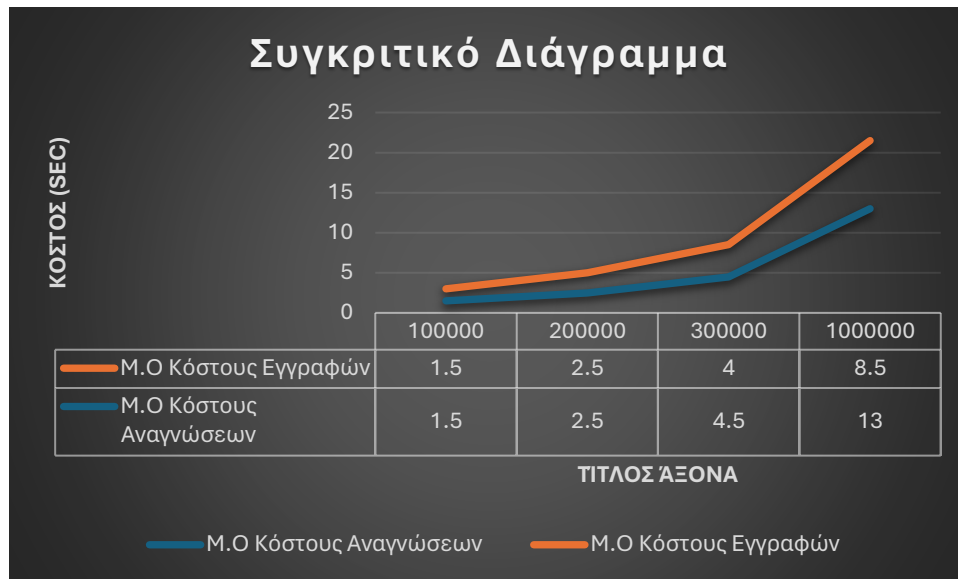
Παρακάτω παρατίθενται γραφικές παραστάσεις για την κάθε λειτουργία για διαφορετικές τιμές των κλειδιών και τα ανάλογα κόστη.



Εικόνα 17



Εικόνα 18



Εικόνα 19

Παρατηρούμε από τα διαγράμματα, ότι όσο αυξάνεται το πλήθος των κλειδιών αυξάνεται και το κόστος των λειτουργιών εγγραφής και ανάγνωσης. Αυτό είναι λογικό, αφού η μηχανή έχει να επεξεργαστεί περισσότερα δεδομένα, με αποτέλεσμα να χρειάζεται παραπάνω χρόνο για να φέρει εις πέρας την δουλειά της. Επίσης στην εικόνα 19 παρατηρούμε ότι η λειτουργία της εγγραφής είναι πιο απαιτητική σε σχέση με την λειτουργία της ανάγνωσης και αυτή η διαφορά κλιμακώνεται όσο αυξάνεται η τιμή των κλειδιών.

Απλή Προσέγγιση

Στο δεύτερο ερώτημα της εργασίας, ζητούμαστε να κάνουμε αλλαγές στον κώδικα ώστε να επιτύχουμε εφαρμογή αμοιβαίου αποκλεισμού στην εκτέλεση των `add` και `get` λειτουργιών προκειμένου να διασφαλιστεί ότι μόνο μία λειτουργία κάθε φορά χρησιμοποιεί τη μηχανή. Επιπλέον, πρέπει να φροντίσουμε την αντιμετώπιση πιθανών συγκρούσεων των νημάτων της εφαρμογής με το νήμα συγχώνευσης της μηχανής που εκτελείται στο υπόβαθρο. Για να το επιτύχουμε αυτό, αρχικά πρέπει να καταφέρουμε τον αμοιβαίο αποκλεισμό στις δύο λειτουργίες. Αυτό το κάναμε με την χρήση `pthread_mutex_t`. Αρχικά αρχικοποιήσαμε ένα `mutex` στην `db.c` (εικόνα 20).

```
9 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

Εικόνα 20

Αυτό το `mutex` που αρχικοποιήσαμε, το χρησιμοποιούμε στις `db_add` και `db_get` ώστε να κάνουμε `lock` και `unlock` τις κρίσιμες περιοχές τους. Το κλειδωμα και το ξεκλειδωμα το επιτυγχάνουμε με χρήση της εντολής `pthread_mutex_lock(&self->mutex);` και `pthread_mutex_unlock(&self->mutex);` (εικόνα 21, 22). Κλειδώνοντας το `mutex` στην αρχή της κρίσιμης περιοχής κάθε μεθόδου, καταφέρνουμε να εκτελείται μόνο μια από τις κρίσιμες περιοχές των δύο λειτουργιών (`db_add` και `db_get`) μέχρι ώτου γίνει `unlock`. Επιπλέον στην `db_get` βάλαμε ένα παραπάνω `unlock` στην περίπτωση που η συνθήκη του `if` ισχύει, ώστε να μην κάνει `return` και παραμένει κλειδωμένο το `mutex`. Επίσης,

συμπεριλαμβάνουμε την `memtable_add` μέσα στην κρίσιμη περιοχή της `db_add` καθώς δεν θέλουμε να αλλάζει η `memtable` και ταυτόχρονα να έχει ξεκινήσει η ανάγνωση της από την `db_get`. Αυτό το κάνουμε με χρήση της εντολής `int result = memtable_add(self->memtable, key, value);` (εικόνα 21). Με αυτόν τον τρόπο αποθηκεύουμε σε μία νέα μεταβλητή, την `result`, ότι επιστρέφει η `memtable_add` και στο τέλος της `db_add` κάνουμε `return` το `result`. Με την ίδια λογική συμπεριλαμβάνουμε την `sst_get` μέσα στην κρίσιμη περιοχή της `db_get`, καθώς δεν θέλουμε ενώ κάνει ανάγνωση στην `sst`, ταυτόχρονα να κάνει `merge`. Αυτό το κάνουμε με χρήση της εντολής `int result = sst_get(self->sst, key, value);` (εικόνα 22). Με αυτόν τον τρόπο αποθηκεύουμε σε μία νέα μεταβλητή, την `result`, ότι επιστρέφει η `sst_get` και στο τέλος της `db_get` κάνουμε `return` το `result`.

```
int db_add(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&self->mutex);
    if (memtable_needs_compaction(self->memtable))
    {
        INFO("Starting compaction of the memtable after %d insertions and %d deletions",
            self->memtable->add_count, self->memtable->del_count);
        sst_merge(self->sst, self->memtable);
        memtable_reset(self->memtable);
        while(pthread_mutex_trylock(&self->sst->cv_lock) == EBUSY){
            ;
        }
        pthread_mutex_unlock(&self->sst->cv_lock);
    }

    int result = memtable_add(self->memtable, key, value);
    pthread_mutex_unlock(&self->mutex);
    return result;
}
```

Εικόνα 21 (`db_add`)

```
int db_get(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&mutex);

    if (memtable_get(self->memtable->list, key, value) == 1){
        pthread_mutex_unlock(&mutex);
        return 1;
    }

    int result = sst_get(self->sst, key, value);
    pthread_mutex_unlock(&mutex);
    return result;
}
```

Εικόνα 22 (`db_get`)

Καθώς εκτελείται η λειτουργία της εγγραφής καλείται η `sst_merge` στην περίπτωση που χρειάζεται `compaction`. Η `sst_merge` όμως ενεργοποιεί το νήμα της συγχώνευσης, το οποίο αναλαμβάνει την διαδικασία της συγχώνευσης. Για να αντιμετωπίσουμε πιθανές συγκρούσεις των νημάτων της εφαρμογής με το νήμα συγχώνευσης της μηχανής που εκτελείται στο υπόβαθρο, χρησιμοποιούμε την `trylock` η οποία ελέγχει αν το `cv_lock`, το οποίο είναι ένα `mutex` της `sst_merge`, είναι ήδη

κλειδωμένο. Αν είναι, δηλαδή η `sst_merge` δεν έχει τελειώσει την διαδικασία της συγχώνευσης, τότε η συνάρτηση ενημερώνει τον χρήστη επιστρέφοντας μια ειδική τιμή (`EBUSY`). Έτσι παραμένει στην `while` όσο εκτελείται η `sst_merge`, χωρίς να αλλάζει τίποτα μέσα στην `while`, και όταν τελειώσει την κάνουμε `unlock` καθώς η `trylock` όταν βρίσκει το `cv_lock` ξεκλειδωτο το κλειδώνει, κάτι που εμείς δεν θέλουμε να συμβαίνει όσο δεν εκτελείται συγχώνευση (εικόνα 23).

```
int db_add(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&self->mutex);
    if (memtable_needs_compaction(self->memtable))
    {
        INFO("Starting compaction of the memtable after %d insertions and %d deletions",
            self->memtable->add_count, self->memtable->del_count);
        sst_merge(self->sst, self->memtable);
        memtable_reset(self->memtable);
        while(pthread_mutex_trylock(&self->sst->cv_lock) == EBUSY){
            ;
        }
        pthread_mutex_unlock(&self->sst->cv_lock);
    }

    int result = memtable_add(self->memtable, key, value);
    pthread_mutex_unlock(&self->mutex);
    return result;
}
```

Εικόνα 23

Εισαγωγή λειτουργίας readwrite και αναγκαίες αλλαγές.

Έπειτα, επεκτείναμε τις δυνατότητες της μηχανής αποθήκευσης δίνοντας την ευκαιρία να προσδιορίσουμε το μίγμα λειτουργιών `add` και `get` καθώς και το ποσοστό από τον κάθε τύπο γράφοντας στο τερματικό την εντολή **`./kiwi-bench readwrite x y z e`**, όπου `x` το συνολικό πλήθος εγγραφών και αναγνώσεων, `y` το ποσοστό του `x` που θα κάνει εγγραφή, `z` το ποσοστό του `x` που θα κάνει ανάγνωση και `e` το πλήθος των νημάτων ανάγνωσης που θα δημιουργηθούν (το `e` θα χρειαστεί μόνο για την περίπτωση της προηγμένης προσέγγισης που θα αναλύσουμε αργότερα. Για τον κώδικα που έχουμε πει ως τώρα, δηλαδή για την απλή προσέγγιση, πληκτρολογούμε `e = 1`, δηλαδή ένα νήμα ανάγνωσης). Για να δουλέψει αυτό, τροποποιήσαμε την `main` της `bench.c` και τις μεθόδους `_write_test` και `_read_test` της `kiwi.c`.

Αρχικά, κάναμε τροποποιήσεις στον κώδικα της `kiwi.c`. Καθώς χρειάζεται να δημιουργήσουμε `threads` για την κάθε λειτουργία, χρειάστηκε να φτιάξουμε ένα `struct` για τα ορίσματα των μεθόδων `_write_test` και `_read_test`, αφού η εντολή που `pthread_create` δεν μας επιτρέπει να βάλουμε πάνω από ένα όρισμα στην μέθοδο που θα υλοποιεί το νήμα. Το `struct`, το οποίο ονομάσαμε `data`, το ορίσαμε στην `bench.h` και περιέχει τις μεταβλητές που χρειάζονται οι μέθοδοι που θα υλοποιούν τα νήματα.

```
//We define a struct, containing the arguments of _write_test and _read_test
typedef struct _data{
    long int count;
    int r;
    DB* db;
}data;
```

Εικόνα 24

Αυτά είναι το long int count, το int r και το DB* db (εικόνα 24). Το struct αυτό είναι ορατό στις μεθόδους της kiwi.c μέσω του #include “bench.h”.

```
//Include bench.h
#include "bench.h"
```

Εικόνα 25

Αφαιρέσαμε από τις μεθόδους _write_test και _read_test τα ορίσματα που είχε, και στην θέση τους βάλαμε void *arg ώστε να παίρνει ως όρισμα τα στοιχεία του struct. Ο λόγος για τον οποίο παίρνει πλέον σαν όρισμα και το db είναι για την πολυνηματική περίπτωση αφού χρειάζεται να γίνονται εγγραφές και αναγνώσεις σε κοινή βάση δεδομένων, και να μην λειτουργεί η κάθε λειτουργία σε διαφορετική βάση.

```
//_write_test now takes a struct as arguments, in order to able to call it with pthread_create
void _write_test(void *arg)
{
    //Struct initialization
    struct _data *d = (struct _data *) arg;
```

Εικόνα 26

```
//_read_test now takes a struct as arguments, in order to able to call it with pthread_create
void _read_test(void *arg)
{
    //Struct initialization
    struct _data *d = (struct _data *) arg;
```

Εικόνα 27

Έπειτα μέσα και στις δύο αυτές μεθόδους, αρχικοποιήσαμε ένα struct d με την εντολή struct _data *d = (struct _data *) arg στο οποίο περνάμε τις τιμές των ορισμάτων με τα οποία καλείται η μέθοδος (εικόνες 26 και 27). Έπειτα αφαιρέσαμε τις εντολές που αρχικοποιούσαν τα ορίσματα, αφού πλέον είναι άχρηστες γιατί τις αρχικοποιούμε στο struct d. Μέσα στις μεθόδους _write_test και _read_test οπουδήποτε χρησιμοποιούσε τα ορίσματα των μεθόδων, πλέον βάλαμε να παίρνει τις τιμές τους από τις ανάλογες μεταβλητές του struct (εικόνες 28 και 29).

```
for (i = 0; i < d->count; i++) {
    if (d->r)
        _random_key(key, KSIZE);
    else
        snprintf(key, KSIZE, "key-%d", i);
    fprintf(stderr, "%d adding %s\n", i, key);
    snprintf(val, VSIZE, "val-%d", i);

    sk.length = KSIZE;
    sk.mem = key;

    sv.length = VSIZE;
    sv.mem = val;

    db_add(d->db, &sk, &sv);
    if ((i % 10000) == 0) {
        fprintf(stderr, "random write finished %d ops%30s\r",
                i,
                "");
        fflush(stderr);
    }
}
```

Εικόνα 28 (_write_test)

```
for (i = 0; i < d->count; i++) {
    memset(key, 0, KSIZE + 1);

    /* if you want to test random write, use the following */
    if (d->r)
        _random_key(key, KSIZE);
    else
        snprintf(key, KSIZE, "key-%d", i);
    fprintf(stderr, "%d searching %s\n", i, key);
    sk.length = KSIZE;
    sk.mem = key;
    ret = db_get(d->db, &sk, &sv);
    if (ret) {
        //db_free_data(sv.mem);
        found++;
    } else {
        INFO("not found key#%s", sk.mem);
    }

    if ((i % 10000) == 0) {
        fprintf(stderr, "random read finished %d ops%30s\r",
                i,
                "");
        fflush(stderr);
    }
}
```

Εικόνα 29(_read_test)

Στην συνέχεια, αφού για την λειτουργία της readwrite πρέπει να κληθούν και η λειτουργία εγγραφής και η λειτουργία ανάγνωσης μαζί, είναι αναγκαίο να γίνεται μία φορά το άνοιγμα και το κλείσιμο της βάσης δεδομένων. Για αυτόν τον λόγο αφαιρέσαμε τις εντολές που είχαν σχέση με την βάση δεδομένων από την Kiwi.c και τις προσθέσαμε στην Bench.c ώστε να γίνεται μία μόνο φορά το άνοιγμα και το κλείσιμο της βάσης. Αυτές οι εντολές που αφαιρέσαμε είναι οι: #define DATAS ("test.db"), db = db_open(DATAS) και η db_close(db).

Τέλος, καθώς θέλουμε να παίρνουμε τα κόστη των λειτουργιών από την bench.c και όχι από την kiwi.c, αφαιρέσαμε τις ανάλογες εντολές και τις ενσωματώσαμε στην main της bench.c. Αυτό το κάναμε γιατί στη περίπτωση που θέλουμε να έχουμε, για παράδειγμα, τρία νήματα για την λειτουργία της ανάγνωσης, να εκτυπώνει στο τέλος πόσο χρόνο χρειάστηκε και για τα τρία νήματα μαζί και όχι για το καθένα ξεχωριστά. Για αυτό τον λόγο και την εκτύπωση των στατιστικών την αναλαμβάνει η bench.c.

Επιπλέον, όπως έχουμε ήδη αναφέρει κάναμε αλλαγές και στην main της bench.c. Η πρώτη προσθήκη ήταν το #include <pthread.h> και το #define DATAS ("test.db"), με σκοπό να δημιουργήσουμε τα νήματα για τις λειτουργίες εγγραφής και ανάγνωσης και για να έχουμε πρόσβαση στην βάση δεδομένων.

```
//Include pthread.h
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

//Define a database to be used
#define DATAS ("testdb")
```

Εικόνα 30

Μέσα στην main αρχικοποιήσαμε τις μεταβλητές long long start,end για τις μετρήσεις του κόστους, την μεταβλητή cost και φτιάξαμε τρία νέα struct _data, το οποίο έχουμε ήδη δημιουργήσει στο bench.h όπως αναφέραμε. Αυτά τα τρία struct θα χρησιμοποιηθούν στην κλήση των μεθόδων για την εγγραφή και την ανάγνωση. Το struct για την εγγραφή το ονομάσαμε add_data και χρειαζόμασταν και δύο struct για την ανάγνωση, για λόγους που θα εξηγήσουμε παρακάτω, των οποίων τα ονόματα είναι get_data και get_data_extra.

```
//Variables for measuring time
long long start,end;
double cost;

//Initializing the structs that we will be used
struct _data add_data;
struct _data get_data;
struct _data get_data_extra;
```

Εικόνα 31

Έπειτα θέσαμε την τιμή start ίση με get_ustime_sec() ώστε να αρχίσει να μετράει την στιγμή που ξεκινάει να τρέχει το πρόγραμμα.

```
//Start measuring system time  
start = get_ustime_sec();
```

Εικόνα 32

Στην περίπτωση που ο χρήστης πληκτρολογήσει write κάναμε τις εξής αλλαγές. Αλλάξαμε την αρχικοποίηση των μεταβλητών r και count και βάλαμε να αρχικοποιούνται οι αντίστοιχες τιμές r και count του struct add_data. Επίσης ανοίγουμε την βάση δεδομένων και την προσθέτουμε στο struct add_data. Ο λόγος που τα προσθέτουμε στο struct και δεν τα αφήνουμε όπως ήταν, είναι για να μπορέσουμε να καλέσουμε την μέθοδο _write_test της kiwi.c με όρισμα το struct, εφόσον όπως αναφέραμε αλλάξαμε τα ορίσματα της συγκεκριμένης μεθόδου και βάλαμε να δέχεται ως όρισμα void* arg. Οπότε στο υπόλοιπο της μεθόδου όπου χρειάζεται να τροποποιήσουμε το r, ή το count ή το db, γράφαμε add_data.r ή add_data.count ή add_data.db αντιστοίχως. Για να τρέξει η λειτουργία εγγραφής καλούμε την μέθοδο write_test με όρισμα το struct add_data και έπειτα κλείνουμε την βάση δεδομένων, αφού έχει ολοκληρωθεί όλη η διαδικασία. Επιπλέον, θέτουμε σε μία μεταβλητή end τον χρόνο την συγκεκριμένη χρονική στιγμή και θέτουμε την μεταβλητή cost = end – start. Το cost είναι το κόστος, δηλαδή ο χρόνος που χρειάστηκε για να ολοκληρώσει την λειτουργία της εγγραφής. Τέλος τυπώνουμε τα στατιστικά των εγγραφών, δηλαδή πόσες εγγραφές έκανε ανά δευτερόλεπτο, την ρυθμιάδοση και το κόστος.

```
if (strcmp(argv[1], "write") == 0) {  
    //Change add data variables  
    add_data.r = 0;  
    add_data.count = atoi(argv[2]);  
  
    //Open the database, since write_test no longer executes db_open()  
    DB* db = db_open(DATAS);  
    add_data.db = db;  
  
    _print_header(add_data.count);  
    _print_environment();  
    if (argc == 4)  
        add_data.r = 1;  
    _write_test(&add_data);  
  
    //Close the database, since _read_test no longer executes db_close()  
    db_close(db);  
  
    //Stop measuring time, calculate cost, and print results  
    end = get_ustime_sec();  
    cost = end - start;  
    printf(LINE);  
    printf("Random-Write (done:%ld): %.6f sec/op; %.1f writes/sec(estimated); cost:%.3f(sec);\n"  
        ,add_data.count, (double)(cost / add_data.count)  
        ,(double)(add_data.count / cost)  
        ,cost);  
}
```

Εικόνα 33

Στην περίπτωση που ο χρήστης πληκτρολογήσει read κάναμε τις εξής αλλαγές. Αλλάξαμε την αρχικοποίηση των μεταβλητών r και count και βάλαμε να αρχικοποιούνται οι αντίστοιχες τιμές r και count του struct get_data. Επίσης ανοίγουμε την βάση δεδομένων και την προσθέτουμε στο struct get_data. Στο υπόλοιπο της μεθόδου όπου χρειάζεται να τροποποιήσουμε το r, ή το count ή το db, γράφαμε get_data.r ή get_data.count ή get_data.db αντιστοίχως. Για να τρέξει η λειτουργία ανάγνωσης καλούμε την μέθοδο read_test με όρισμα το struct get_data και έπειτα κλείνουμε την βάση

δεδομένων, αφού έχει ολοκληρωθεί όλη η διαδικασία. Επιπλέον, θέτουμε σε μία μεταβλητή end τον χρόνο την συγκεκριμένη χρονική στιγμή και θέτουμε την μεταβλητή cost = end – start. Το cost είναι το κόστος, δηλαδή ο χρόνος που χρειάστηκε για να ολοκληρώσει την λειτουργία της ανάγνωσης. Τέλος τυπώνουμε τα στατιστικά των αναγνώσεων, δηλαδή πόσες αναγνώσεις έκανε ανά δευτερόλεπτο, την ρυθμαπόδοση και το κόστος.

```
} else if (strcmp(argv[1], "read") == 0) {  
    //Change get_data variables  
    get_data.r = 0;  
    get_data.count = atoi(argv[2]);  
  
    //Open database, since read test no longer executes db_open()  
    DB* db = db_open(DATAS);  
    get_data.db = db;  
  
    _print_header(get_data.count);  
    _print_environment();  
    if (argc == 4)  
        get_data.r = 1;  
  
    _read_test(&get_data);  
  
    //Close database, since _read_test no longer executes db_close()  
    db_close(db);  
  
    //Stop measuring time, calculate cost, and print results  
    end = get_ustime_sec();  
    cost = end - start;  
    printf(LINE);  
    printf("|Random-Read    (done:%ld): %.6f sec/op; %.1f reads /sec(estimated); cost:%.3f(sec)\n",  
        get_data.count,  
        (double)(cost / get_data.count),  
        (double)(get_data.count / cost),  
        cost);  
}
```

Εικόνα 34

Τέλος προσθέσαμε και την περίπτωση που ο χρήστης πληκτρολογήσει readwrite.

```
} else if (strcmp(argv[1], "readwrite") == 0) {
```

Εικόνα 35

Σε αυτήν την περίπτωση ξανά, αλλάξαμε την αρχικοποίηση της μεταβλητής r και βάλαμε να αρχικοποιούνται οι αντίστοιχες τιμές r των struct add_data, get_data και get_data_extra.

```
//Change struct's variables  
add_data.r = 0;  
get_data.r = 0;  
get_data_extra.r = 0;
```

Εικόνα 36

Παίρνουμε από το τερματικό, με την βοήθεια της atoi(argv[x]), τα ποσοστά εγγραφών και αναγνώσεων που ζητάει ο χρήστης να εκτελεστούν και τα αποθηκεύουμε στις μεταβλητές int percentageW για το ποσοστό των εγγραφών και percentageR για το ποσοστό των αναγνώσεων. Επίσης θέτουμε σε μία μεταβλητή int get_threads_count τον αριθμό των νημάτων που θα εκτελέσουν αναγνώσεις. Την τιμή αυτή την παίρνουμε επίσης από το τερματικό.

```
//Store command line arguments from user
int percentageW = atoi(argv[3]);
int percentageR = atoi(argv[4]);
int get_threads_count = atoi(argv[5]);
```

Εικόνα 37

Έπειτα ανοίγουμε μια βάση δεδομένων και ενημερώνουμε τα τρία struct μας με την βάση που ανοίξαμε ώστε και η λειτουργία της εγγραφής και της ανάγνωσης να δουλεύουν σε κοινή βάση.

```
//Open the database and update the database of the structs, so that write and read operations
//will be executed on the same database
DB* db = db_open(DATAS);
add_data.db = db;
get_data.db = db;
get_data_extra.db = db;
```

Εικόνα 38

Κρατάμε σε μία μεταβλητή count των αριθμό των συνολικών εκτελέσεων, από το τερματικό, δηλαδή και εγγραφές και αναγνώσεις.

```
//Store how many operations to be executed
long int count = atoi(argv[2]);
```

Εικόνα 39

Θέτουμε το count στο struct της add ίσο με $\text{count} * \text{percentageW} / 100$ και το count στο struct της get ίσο με $\text{count} * \text{percentageR} / 100$.

```
//Calculate how many write and how many read operations will be executed
add_data.count = count*percentageW/100;
get_data.count = count*percentageR/100;
```

Εικόνα 40

Αρχικοποιήσαμε το id του νήματος της εγγραφής με pthread_t add_thread και τα id των νημάτων ανάγνωσης με pthread_t get_thread[get_threads_count].

```
//Initialize the thread ids
pthread_t add_thread;
pthread_t get_thread[get_threads_count];
```

Εικόνα 41

Μετά, δημιουργήσαμε το νήμα της εγγραφής με την εντολή: pthread_create(&add_thread, NULL, (void (*)(void *)) _write_test, (void*)&add_data) η οποία εκτελεί την μέθοδο _write_test με όρισμα το struct add_data.

```
//Create the write thread, which will call _write_test with add_data struct as argument
pthread_create(&add_thread, NULL, (void (*)(void *)) _write_test, (void *) &add_data);
```

Εικόνα 42

Με σκοπό να είναι ορατή η μέθοδος `_write_test` κατά την δημιουργία του νήματος την έχουμε δηλώσει ακριβώς πριν την `main` με το `void _write_test(void *arg)`.

```
//Initialize _write_test and _read_test, so that pthread_create can use them as arguments
void _write_test(void *arg);
void _read_test(void *arg);
```

Εικόνα 43

Για την δημιουργία των νημάτων ανάγνωσης αρχικά ελέγχουμε αν ο χρήστης ζητάει ένα ή παραπάνω νήματα ανάγνωσης. Στην περίπτωση του ενός νήματος καλούμε την `pthread_create(&get_thread[0],NULL,(void (*)(void*)) _read_test, (void*) &get_data)` η οποία εκτελεί την μεθόδο `_read_test` με όρισμα το `struct get_data`. Με σκοπό να είναι ορατή η μέθοδος `_read_test` κατά την δημιουργία του νήματος την έχουμε δηλώσει ακριβώς πριν την `main` με το `void _read_test(void *arg)` (εικόνα 43). Αλλά όταν ο χρήστης επιλέξει να υπάρχουν παραπάνω από ένα νήματα, τότε χρειάζεται να διαμοιράσουμε τον αριθμό των αναγνώσεων με δίκαιο τρόπο. Για τον σκοπό αυτό, ενημερώνουμε το `get_data.count` με `get_data.count / get_threads_count` το οποίο είναι πόσες αναγνώσεις πρέπει να κάνει το κάθε νήμα. Επειδή όμως υπάρχει περίπτωση το πηλίκο της διαίρεσης να μην είναι ακέραιος, δηλαδή να μην διαμοιράζονται ακριβώς οι αναγνώσεις, καλούμε τα πρώτα `get_threads_count - 1` νήματα με το ακέραιο μέρος της διαίρεσης, με μία `for` και την εντολή `pthread_create(&get_thread[0],NULL,(void (*)(void*)) _read_test, (void*) &get_data)`, και έπειτα στο ένα νήμα που δεν έχουμε δημιουργήσει ακόμα, το καλούμε με το υπόλοιπο της διαίρεσης.

```
//Check if there will be more than one add threads
//If so, split the add operations fairly, to each thread
if(get_threads_count > 1){
    //Get the integer part of the division
    get_data.count = get_data.count / get_threads_count;

    //Create every thread except one, where its one calls _read_test with get_data as argument,
    //and its read test will execute the integer part of the get_data.count = get_data.count / get_threads_count division
    for(int i = 0; i < get_threads_count-1; i++){
        pthread_create(&get_thread[i],NULL,(void (*)(void*)) _read_test, (void *) &get_data);
    }

    //Calculate the remaining adds, update get_data_extra struct, and create the last add thread
    get_data_extra.count = count*percentageR/100 - (get_data.count*(get_threads_count-1));
    pthread_create(&get_thread[get_threads_count-1],NULL,(void (*)(void*)) _read_test, (void *) &get_data_extra);
}

//If there is only one add thread, create it and call read test with get_data struct as argument
else{
    pthread_create(&get_thread[0],NULL,(void (*)(void*)) _read_test, (void *) &get_data);
}
```

Εικόνα 44

Με αυτόν τον τρόπο, καταφέρνουμε να διαμοιράζουμε σε όλα τα νήματα δίκαια τις αναγνώσεις. Τέλος, κάνουμε `join` το ένα νήμα της εγγραφής με την εντολή `pthread_join(add_thread,NULL)`; και με μία `for` κάνουμε `join` όλα τα νήματα των αναγνώσεων με την εντολή `pthread_join(get_thread[i],NULL)`.


```
//Join all the threads, after they are done working
pthread_join(add_thread, NULL);
for(int i = 0; i < get_threads_count; i++){
    pthread_join(get_thread[i], NULL);
}
```

Εικόνα 45

Έπειτα, κλείνουμε την βάση δεδομένων, αφού έχει ολοκληρωθεί όλη η διαδικασία.

```
//Close the database
db_close(db);
```

Εικόνα 46

Επιπλέον, θέτουμε σε μία μεταβλητή end τον χρόνο την συγκεκριμένη χρονική στιγμή και θέτουμε την μεταβλητή cost = end – start. Το cost είναι το κόστος, δηλαδή ο χρόνος που χρειάστηκε για να ολοκληρώσει την λειτουργία των εγγραφών και των αναγνώσεων. Τέλος τυπώνουμε τα στατιστικά των εγγραφών και των αναγνώσεων, δηλαδή πόσο χρόνο χρειάζεται ώστε να εκτελεστεί μια ανάγνωση ή μια εγγραφή.

```
//Stop measuring time, calculate cost, and print results
end = get_ustime_sec();
cost = end - start;
printf(LINE);
printf("|Random-Read_Write      (done:%ld): %.6f sec/op; cost:%.3f(sec)\n",
       count,
       (double)(cost /count),
       cost);
```

Εικόνα 47

Προηγμένη Προσέγγιση

Για την προηγμένη προσέγγιση σκεφτήκαμε να την υλοποιήσουμε με σημαφόρους. Για αρχή όποια νέα αλλαγή κάναμε είναι στην db.c. Κάναμε #include <semaphore.h> ώστε να έχουμε πρόσβαση στις λειτουργίες των σημαφόρων.

```
//Include semaphore.h
#include <semaphore.h>
```

Εικόνα 48

Δηλώσαμε την σημαφόρο mutual_exclusion με την εντολή sem_t mutual_exclusion. Επίσης, δηλώσαμε τον ακέραιο mutual_value στον οποίο αποθηκεύουμε την τιμή της σημαφόρου. Επίσης, αρχικοποιήσαμε τον ακέραιο i ίσο με 0, του οποίου την χρησιμότητα θα εξηγήσουμε αργότερα.

```
//Initilize the semaphore, a variable to store its value, and a flag to be used later
sem_t mutual_exclusion;
int mutual_value;
int i = 0;
```

Εικόνα 49

Επόμενη προσθήκη στον κώδικα είναι στην db_open όπου αρχικοποιούμε την σημαφόρο mutual_exclusion με την εντολή sem_init(&mutual_exclusion,0,1), όπου το 0 συμβολίζει ότι δεν θέλουμε να μοιράζεται fork διεργασίες και θέτουμε την τιμή του σε 1. Η τιμή 1 σημαίνει ότι θέλουμε κάθε φορά να τρέχει μια εργασία, είτε μία add είτε μία ή περισσότερες get. Ο λόγος που το αρχικοποιούμε στην db_open είναι ότι αφού αυτή η μέθοδος καλείται κάθε φορά όταν ξεκινάει το πρόγραμμα τότε θα αρχικοποιείται και η σημαφόρος.

```
//Initialize the semaphore with 0, since we dont want it to share fork() processes
//and 1 so that either write or read operation is running
sem_init(&mutual_exclusion, 0, 1);
```

Εικόνα 50

Για την db_add καλούμε sem_wait(&mutual_exclusion) στην αρχή της και sem_post(&mutual_exclusion) στο τέλος της. Κάθε φορά που καλείται μια add μειώνεται η τιμή του σημαφόρου σε 0 μέσω της sem_wait(), υποδεικνύοντας ότι η λειτουργία εγγραφής είναι ενεργή.

```
//Reduce the value of the semaphore by 1, so that it becomes 0
sem_wait(&mutual_exclusion);
```

Εικόνα 51

Στο τέλος, εφόσον έχει τελειώσει η add, η τιμή της σημαφόρου αυξάνεται κατά 1 μέσω της sem_post() υποδεικνύοντας ότι η εγγραφή τελείωσε.

```
//Incrrement the value of the semaphore and set it back to 1
sem_post(&mutual_exclusion);
```

Εικόνα 52

Με αυτόν τον τρόπο όσο τρέχει η add δεν υπάρχει κίνδυνος να γίνει η τιμή της σημαφόρου αρνητική με αποτέλεσμα να μπει η διεργασία στο queue της σημαφόρου και να παγώσει εκεί, εφόσον οι τιμές κυμαίνονται μεταξύ του 0 και του 1. Επιπλέον συμπεριλάβαμε την memtable_add μέσα στην κρίσιμη περιοχή της db_add καθώς δεν θέλουμε να αλλάζει η memtable και ταυτόχρονα να έχει ξεκινήσει η ανάγνωση της από την db_get. Αυτό το κάνουμε με χρήση της εντολής int result = memtable_add(self->memtable, key, value).

```
//Store the return of memtable_add, so that memtable_add is executed before db_add is finished
int result = memtable_add(self->memtable, key, value);
```

Εικόνα 53

Με αυτόν τον τρόπο αποθηκεύουμε σε μία νέα μεταβλητή, την result, ότι επιστρέφει η memtable_add και στο τέλος της db_add κάνουμε return το result.

`return result;`

Εικόνα 54

Καθώς εκτελείται η λειτουργία της εγγραφής καλείται η `sst_merge` στην περίπτωση που χρειάζεται `compaction`. Η `sst_merge` όμως ενεργοποιεί το νήμα της συγχώνευσης, το οποίο αναλαμβάνει την διαδικασία της συγχώνευσης. Για να αντιμετωπίσουμε πιθανές συγκρούσεις των νημάτων της εφαρμογής με το νήμα συγχώνευσης της μηχανής που εκτελείται στο υπόβαθρο, χρησιμοποιούμε την `trylock` η οποία ελέγχει αν το `cv_lock`, το οποίο είναι ένα `mutex` της `sst_merge`, είναι ήδη κλειδωμένο. Αν είναι, δηλαδή η `sst_merge` δεν έχει τελειώσει την διαδικασία της συγχώνευσης, τότε η συνάρτηση ενημερώνει τον χρήστη επιστρέφοντας μια ειδική τιμή (`EBUSY`). Έτσι παραμένει στην `while` όσο εκτελείται η `sst_merge`, χωρίς να αλλάζει τίποτα μέσα στην `while`, και όταν τελειώσει την κάνουμε `unlock` καθώς η `trylock` όταν βρίσκει το `cv_lock` ξεκλειδωτο το κλειδώνει, κάτι που εμείς δεν θέλουμε να συμβαίνει όσο δεν εκτελείται συγχώνευση.

```
//Wait until the merge thread is finished and after it's done, unlock the cv_lock,
//since try_lock locks the lock
while(pthread_mutex_trylock(&self->sst->cv_lock) == EBUSY){
    ;
}
pthread_mutex_unlock(&self->sst->cv_lock);
```

Εικόνα 55

Για την `db_get`, αρχικά, με την εντολή `sem_getvalue(&mutual_exclusion,&mutual_value)` ενημερώνουμε την τιμή του `mutual_value` με την τιμή της σημαφόρου, έτσι ώστε όσο ισούται με 0, δηλαδή όσο τρέχει η `add`, να παραμένει παγωμένο το ή τα νήματα που εκτελούν αναγνώσεις. Αυτό το επιτυγχάνουμε με χρήση της `while` και την κλήση της `sem_getvalue()` έτσι ώστε να ενημερώνεται όσο τρέχει η `while`, και στην περίπτωση που γίνει 1 να “βγει” από αυτή.

```
//Update the mutual_value with the value of the semaphore
//so that while can check if it is equal to 0.
//If it is 0, that means that the write thread is running,
//so any read thread should wait
sem_getvalue(&mutual_exclusion,&mutual_value);
while(mutual_value == 0){
    sem_getvalue(&mutual_exclusion,&mutual_value);
}
```

Εικόνα 56

Στην συνέχεια, με έλεγχο της `if` αν η τιμή του `i` είναι ίση με 0, το οποίο δηλώνει ότι ξεκίνησε μία λειτουργία ανάγνωσης από ένα από τα νήματα, τότε τίθεται ίσο με 1, καλείται η `sem_wait(&mutual_exclusion)` μειώνοντας έτσι κατά 1 την τιμή της σημαφόρου και ενημερώνουμε την τιμή του `mutual_value` μέσω της `sem_getvalue(&mutual_exclusion,&mutual_value)`. Αυτός ο έλεγχος θέλουμε να γίνεται από μόνο ένα νήμα ανάγνωσης την φορά καθώς αν όλα τα νήματα ανάγνωσης καλούσαν την `sem_wait()` θα έβρισκαν την τιμή αρνητική, θα έμπαιναν στην `queue` της σημαφόρου και έτσι δεν θα εκτελούνταν ταυτόχρονα αλλά θα έπρεπε να βγουν σειριακά από την ουρά.

```
//While a read thread is running, the write thread shouldn't run,  
//so we call sem_wait and set the semaphore value to 0.  
//That way, when the write thread calls db_add and it executes sem_wait  
//the value becomes -1 and the add operation is added in the queue of the semaphore,  
//waiting for the get operations to be done  
if(i == 0){  
    i = 1;  
    sem_wait(&mutual_exclusion);  
    sem_getvalue(&mutual_exclusion,&mutual_value);  
}
```

Εικόνα 57

Όταν τελειώνει η λειτουργία της get, δηλαδή είτε όταν η τιμή του κλειδιού βρίσκεται στο memtable είτε στο sst, εκτελούμε ανάλογο έλεγχο για το αν το i είναι ίσο με 1 όπου τότε το ξανά θέτουμε το i στο 0, ώστε να δηλώσει ότι τελείωσε η ανάγνωση, αυξάνουμε την τιμή της σημαφόρου στο 1 μέσω της sem_post(&mutual_exclusion) και ξανά ενημερώνουμε την τιμή του mutual_value μέσω της sem_getvalue(&mutual_exclusion,&mutual_value).

```
//After the read threads are done, they increment the value of the semaphore,  
//so that the write thread that is waiting in the queue, gets released  
if(i ==1){  
    i = 0;  
    sem_post(&mutual_exclusion);  
    sem_getvalue(&mutual_exclusion,&mutual_value);  
}
```

Εικόνα 58

Επίσης, συμπεριλαμβάνουμε την sst_get μέσα στην κρίσιμη περιοχή της db_get, καθώς δεν θέλουμε ενώ κάνει ανάγνωση στην sst, ταυτόχρονα να κάνει merge. Αυτό το κάνουμε με χρήση της εντολής int result = sst_get(self->sst, key, value).

```
//Store the return of the sst_get, so that sst_get gets executed before db_get is finished  
int result = sst_get(self->sst, key, value);
```

Εικόνα 59

Με αυτόν τον τρόπο αποθηκεύουμε σε μία νέα μεταβλητή, την result, ότι επιστρέφει η sst_get και στο τέλος της db_get κάνουμε return to result.

```
return result;
```

Εικόνα 60

Κάπου εδώ να προσθέσουμε ότι η λειτουργία της get δεν είναι 100% αξιόπιστη καθώς γνωρίζουμε ότι μερικές φορές πέφτει σε ατέρμονο βρόχο στην πρώτη while της db_get. Αυτό συμβαίνει γιατί, ανάλογα με τις ταχύτητες του συστήματος και του πόσο γρήγορα εκτελούνται οι εντολές, ιδανικά ο έλεγχος των if για την τιμή του i θέλουμε να γίνεται από ένα από την νήματα, που τρέχουν εκείνη την στιγμή, την κάθε φορά. Στην περίπτωση που ο αρχικός έλεγχος, αν το i ισούται με 0, τυγχάνει να γίνει ταυτόχρονα από παραπάνω από ένα νήματα, έστω γ νήματα, και δεν προλάβει να γίνει ο δεύτερος έλεγχος, δηλαδή αν το i ισούται με 1, και από τα γ νήματα τότε η τιμή του σημαφόρου παραμένει 0 και μπαίνει σε ατέρμονο βρόχο. Αυτή η περίπτωση δεν είναι σύνηθες το να συμβεί απλά θεωρήσαμε σωστό να το αναφέρουμε.

Πειραματικές Μετρήσεις

Παρακάτω, παρατίθενται στιγμιότυπα οθόνης από τις μετρήσεις. Για να λάβουμε τα αποτελέσματα των εγγραφών, χρησιμοποιήσαμε την εντολή `./kiwi-bench write x`, όπου `x` ο αριθμός των κλειδιών για τον οποίο θέλουμε να κάνει την μέτρηση κάθε φορά. Αντίστοιχα, για να παρατηρήσουμε τα κόστη των αναγνώσεων χρησιμοποιήσαμε την εντολή `./kiwi-bench read x`, όπου `x` ο αριθμός των κλειδιών για τον οποίο θέλουμε να κάνει την μέτρηση κάθε φορά. Επιπλέον, για να παρατηρήσουμε τα κόστη των `readwrite`, δηλαδή μίγμα των λειτουργιών εγγραφής και ανάγνωσης χρησιμοποιήσαμε την εντολή `./kiwi-bench readwrite x y z e`, όπου `x` το συνολικό πλήθος εγγραφών και αναγνώσεων, `y` το ποσοστό του `x` που θα κάνει εγγραφή, `z` το ποσοστό του `x` που θα κάνει ανάγνωση και `e` το πλήθος των νημάτων ανάγνωσης που θα δημιουργηθούν (στην περίπτωση της απλής προσέγγισης πληκτρολογούμε `e = 1`).

Μετρήσεις Απλής Προσέγγισης

Τα αποτελέσματα των μετρήσεων που ακολουθούν έγιναν με τον κώδικα που αναλύσαμε στην περίπτωση της απλής προσέγγισης. Τα συγκεκριμένα κομμάτια κώδικα έχουν παραμείνει στο `db.c` αρχείο, αλλά είναι σε σχόλια και έχουν σημαδευτεί με σχόλια που ξεκαθαρίζουν ότι πρόκειται για την απλή προσέγγιση.

Κόστος εγγραφών εκατό χιλιάδων ($x = 100.000$) κλειδιών:

```
Random-Write (done:100000): 0.000010 sec/op; 100000.0 writes/sec(estimated); cost:1.000(sec);
```

Εικόνα 61

Τρέξαμε το πρόγραμμα για 100.000 εγγραφές και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 100.000 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 1 δευτερόλεπτο (εικόνα 61).

Κόστος αναγνώσεων εκατό χιλιάδων ($x = 100.000$) κλειδιών:

```
Random-Read (done:100000): 0.000010 sec/op; 100000.0 reads /sec(estimated); cost:1.000(sec)
```

Εικόνα 62

Τρέξαμε το πρόγραμμα για 100.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 100.000 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 1 δευτερόλεπτο (εικόνα 62).

Κόστος εγγραφών διακοσίων χιλιάδων ($x = 200.000$) κλειδιών:

```
|Random-Write (done:200000): 0.000015 sec/op; 66666.7 writes/sec(estimated); cost:3.000(sec);
```

Εικόνα 63

Τρέξαμε το πρόγραμμα για 200.000 εγγραφές και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 66.666,7 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 3 δευτερόλεπτα (εικόνα 63).

Κόστος αναγνώσεων διακοσίων χιλιάδων ($x = 200.000$) κλειδιών:

```
Random-Read (done:200000): 0.000010 sec/op; 100000.0 reads /sec(estimated); cost:2.000(sec)
```

Εικόνα 64

Τρέξαμε το πρόγραμμα για 200.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 100.000 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 2 δευτερόλεπτα (εικόνα 64).

Κόστος εγγραφών τριακοσίων χιλιάδων ($x = 300.000$) κλειδιών:

```
Random-Write (done:300000): 0.000017 sec/op; 60000.0 writes/sec(estimated); cost:5.000(sec);
```

Εικόνα 65

Τρέξαμε το πρόγραμμα για 300.000 εγγραφές και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 60.000 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 5 δευτερόλεπτα (εικόνα 65).

Κόστος αναγνώσεων τριακοσίων χιλιάδων ($x = 300.000$) κλειδιών:

```
Random-Read (done:300000): 0.000010 sec/op; 100000.0 reads /sec(estimated); cost:3.000(sec)
```

Εικόνα 66

Τρέξαμε το πρόγραμμα για 300.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 100.000 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 3 δευτερόλεπτα (εικόνα 66).

Κόστος εγγραφών ενός εκατομμυρίου ($x = 1.000.000$) κλειδιών:

```
Random-Write (done:1000000): 0.000015 sec/op; 66666.7 writes/sec(estimated); cost:15.000(sec);
```

Εικόνα 67

Τρέξαμε το πρόγραμμα για 1.000.000 εγγραφές και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 66.666,7 εγγραφές / δευτερόλεπτο και το κόστος ίσο με 15 δευτερόλεπτα (εικόνα 67).

Κόστος αναγνώσεων ενός εκατομμυρίου ($x = 1.000.000$) κλειδιών:

```
Random-Read (done:1000000): 0.000009 sec/op; 111111.1 reads /sec(estimated); cost:9.000(sec)
```

Εικόνα 68

Τρέξαμε το πρόγραμμα για 1.000.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: η ρυθμαπόδοση ίση με 111.111,1 αναγνώσεις / δευτερόλεπτο και το κόστος ίσο με 9 δευτερόλεπτα (εικόνα 68).



Εικόνα 69



Εικόνα 70



Εικόνα 71

Παρατηρούμε από τα διαγράμματα, ότι όσο αυξάνεται το πλήθος των κλειδιών αυξάνεται και το κόστος των λειτουργιών εγγραφής και ανάγνωσης. Αυτό είναι λογικό, αφού η μηχανή έχει να επεξεργαστεί περισσότερα δεδομένα, με αποτέλεσμα να χρειάζεται παραπάνω χρόνο για να φέρει εις πέρας την δουλειά της. Παρόλα αυτά, δεν παρατηρούμε μεγάλες αποκλίσεις από τις τιμές των κοστών πριν τις αλλαγές στον κώδικα. Επίσης στην εικόνα 71 παρατηρούμε ότι η λειτουργία της εγγραφής είναι πιο απαιτητική σε σχέση με την λειτουργία της ανάγνωσης και αυτή η διαφορά κλιμακώνεται όσο αυξάνεται η τιμή των κλειδιών.

Κόστος readwrite διακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 50% (100.000) και ποσοστό αναγνώσεων 50% (100.000) και με 1 νήμα ανάγνωσης:

```
|Random-Read Write (done:200000): 0.000015 sec/op; cost:3.000(sec)
```

Εικόνα 72

Τρέξαμε το πρόγραμμα για 100.000 εγγραφές και 100.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 3 δευτερόλεπτα και ότι χρειάζεται 15 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 72).

Κόστος readwrite τετρακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 50% (200.000) και ποσοστό αναγνώσεων 50% (200.000) και με 1 νήμα ανάγνωσης:

```
|Random-Read_Write (done:400000): 0.000013 sec/op; cost:5.000(sec)
```

Εικόνα 73

Τρέξαμε το πρόγραμμα για 200.000 εγγραφές και 200.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 5 δευτερόλεπτα και ότι χρειάζεται 13μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 73).

Κόστος readwrite δύο εκατομμυρίων κλειδιών με ποσοστό εγγραφών 50% (1.000.000) και ποσοστό αναγνώσεων 50% (1.000.000) και με 1 νήμα ανάγνωσης:

```
Random-Read_Write (done:2000000): 0.000013 sec/op; cost:26.000(sec)
```

Εικόνα 74

Τρέξαμε το πρόγραμμα για 1.000.000 εγγραφές και 1.000.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 26 δευτερόλεπτα και ότι χρειάζεται 13μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 74).



Εικόνα 75

Παρατηρούμε από το διάγραμμα ότι ενώ φαίνεται για μικρές τιμές κλειδιών να μην υπάρχει κάποια διαφορά στα κόστη των δύο υλοποιήσεων, για 2 εκατομμύρια αναγνώσεις και εγγραφές συνολικά η εναλλαγή μεταξύ των δύο λειτουργιών χρησιμοποιώντας νήματα φαίνεται να μην συμβάλει στην πιο γρήγορη εκτέλεση.

Κόστος readwrite τετρακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 70% (280.000) και ποσοστό αναγνώσεων 30% (120.000) και με 1 νήμα ανάγνωσης:

```
Random-Read_Write (done:400000): 0.000013 sec/op; cost:5.000(sec)
```

Εικόνα 76

Τρέξαμε το πρόγραμμα για 280.000 εγγραφές και 120.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 5 δευτερόλεπτα και ότι χρειάζεται 13μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 76).

Κόστος readwrite τετρακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 30% (120.000) και ποσοστό αναγνώσεων 70% (280.000) και με 1 νήμα ανάγνωσης:

```
Random-Read_Write (done:400000): 0.000010 sec/op; cost:4.000(sec)
```

Εικόνα 77

Τρέξαμε το πρόγραμμα για 120.000 εγγραφές και 280.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 4 δευτερόλεπτα και ότι χρειάζεται 10 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 77).

Κόστος readwrite δύο εκατομμυρίων κλειδιών με ποσοστό εγγραφών 70% (1.400.000) και ποσοστό αναγνώσεων 30% (600.000) και με 1 νήμα ανάγνωσης:

```
Random-Read_Write (done:2000000): 0.000015 sec/op; cost:29.000(sec)
```

Εικόνα 78

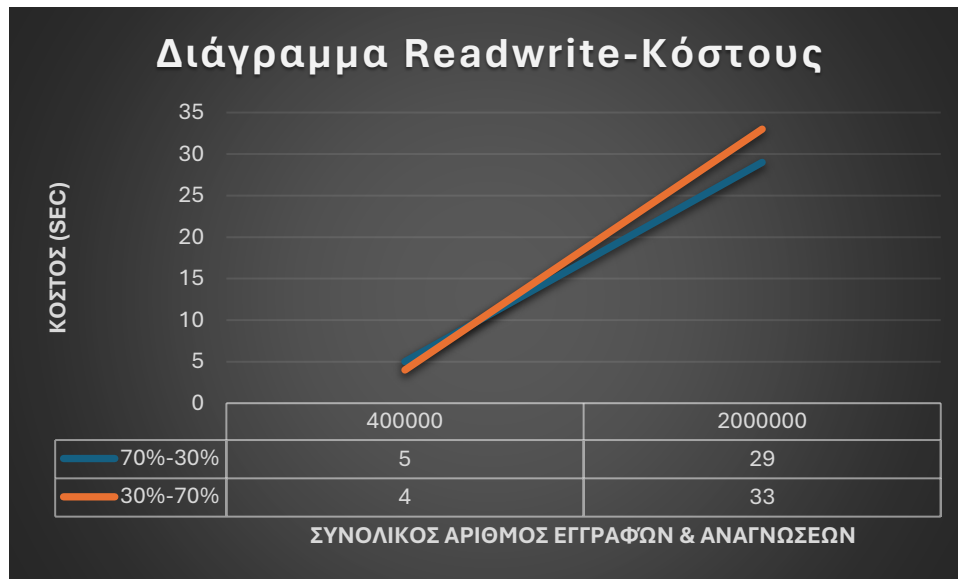
Τρέξαμε το πρόγραμμα για 1.400.000 εγγραφές και 600.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 29 δευτερόλεπτα και ότι χρειάζεται 15μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 78).

Κόστος readwrite δύο εκατομμυρίων κλειδιών με ποσοστό εγγραφών 30% (600.000) και ποσοστό αναγνώσεων 70% (1.400.000) και με 1 νήμα ανάγνωσης:

```
Random-Read_Write (done:2000000): 0.000017 sec/op; cost:33.000(sec)
```

Εικόνα 79

Τρέξαμε το πρόγραμμα για 600.000 εγγραφές και 1.400.000 αναγνώσεις και τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 33 δευτερόλεπτα και ότι χρειάζεται 17μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 79).



Εικόνα 80

Μετρήσεις Προηγμένης Προσέγγισης

Στις μετρήσεις για την προηγμένη λύση χρησιμοποιήσαμε μόνο την εντολή `./kiwi-bench readwrite x y z e`, όπου `x` το συνολικό πλήθος εγγραφών και αναγνώσεων, `y` το ποσοστό του `x` που θα κάνει εγγραφή, `z` το ποσοστό του `x` που θα κάνει ανάγνωση και `e` το πλήθος των νημάτων ανάγνωσης που θα δημιουργηθούν. Ο λόγος είναι ότι θεωρήθηκε άσκοπο να κληθούν οι `write` και `read` αφού δεν μπορούμε να εξάγουμε αποτελέσματα που δεν έχουμε ήδη συμπεριλάβει στην αναφορά.

Κόστος readwrite διακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 50% (100.000) και ποσοστό αναγνώσεων 50% (100.000) και με 3 νήματα ανάγνωσης:

```
Random-Read_Write (done:200000): 0.000010 sec/op; cost:2.000(sec)
```

Εικόνα 81

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 2 δευτερόλεπτα και ότι χρειάζεται 10 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 81).

Κόστος readwrite διακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 50% (100.000) και ποσοστό αναγνώσεων 50% (100.000) και με 5 νήματα ανάγνωσης:

```
Random-Read_Write (done:200000): 0.000010 sec/op; cost:2.000(sec)
```

Εικόνα 82

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 7 δευτερόλεπτα και ότι χρειάζεται 35 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 82).

Κόστος readwrite τετρακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 50% (200.000) και ποσοστό αναγνώσεων 50% (200.000) και με 3 νήματα ανάγνωσης:

```
Random-Read_Write      (done:400000): 0.000015 sec/op; cost:6.000(sec)
```

Εικόνα 83

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 6 δευτερόλεπτα και ότι χρειάζεται 15 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 83).

Κόστος readwrite τετρακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 50% (200.000) και ποσοστό αναγνώσεων 50% (200.000) και με 5 νήματα ανάγνωσης:

```
|Random-Read_Write      (done:400000): 0.000015 sec/op; cost:6.000(sec)
```

Εικόνα 84

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 6 δευτερόλεπτα και ότι χρειάζεται 15 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 84).

Κόστος readwrite διακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 70% (140.000) και ποσοστό αναγνώσεων 30% (60.000) και με 3 νήματα ανάγνωσης:

```
|Random-Read_Write      (done:200000): 0.000015 sec/op; cost:3.000(sec)
```

Εικόνα 85

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 3 δευτερόλεπτα και ότι χρειάζεται 15 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 85).

Κόστος readwrite διακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 70% (140.000) και ποσοστό αναγνώσεων 30% (60.000) και με 5 νήματα ανάγνωσης:

```
-----+-----+-----+-----+
Random-Read Write      (done:200000): 0.000010 sec/op; cost:2.000(sec)
```

Εικόνα 86

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 2 δευτερόλεπτα και ότι χρειάζεται 10 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 86).

Κόστος readwrite διακοσίων χιλιάδων κλειδίων με ποσοστό εγγραφών 30% (60.000) και ποσοστό αναγνώσεων 70% (140.000) και με 3 νήματα ανάγνωσης:

```
|Random-Read Write      (done:200000): 0.000010 sec/op; cost:2.000(sec)
```

Εικόνα 87

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 2 δευτερόλεπτα και ότι χρειάζεται 10 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 87).

Κόστος readwrite διακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 30% (60.000) και ποσοστό αναγνώσεων 70% (140.000) και με 5 νήματα ανάγνωσης:

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Random-Read_Write      (done:200000): 0.000010 sec/op; cost:2.000(sec)|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Εικόνα 88

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 2 δευτερόλεπτα και ότι χρειάζεται 10 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 88).

Κόστος readwrite τετρακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 70% (280.000) και ποσοστό αναγνώσεων 30% (120.000) και με 3 νήματα ανάγνωσης:

```
|Random-Read_Write      (done:400000): 0.000015 sec/op; cost:6.000(sec)|
```

Εικόνα 89

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 6 δευτερόλεπτα και ότι χρειάζεται 15 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 89).

Κόστος readwrite τετρακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 70% (280.000) και ποσοστό αναγνώσεων 30% (120.000) και με 5 νήματα ανάγνωσης:

```
|Random-Read_Write      (done:400000): 0.000023 sec/op; cost:9.000(sec)|
```

Εικόνα 90

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 9 δευτερόλεπτα και ότι χρειάζεται 23 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 90).

Κόστος readwrite τετρακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 30% (120.000) και ποσοστό αναγνώσεων 70% (280.000) και με 3 νήματα ανάγνωσης:

```
|Random-Read_Write      (done:400000): 0.000020 sec/op; cost:8.000(sec)|
```

Εικόνα 91

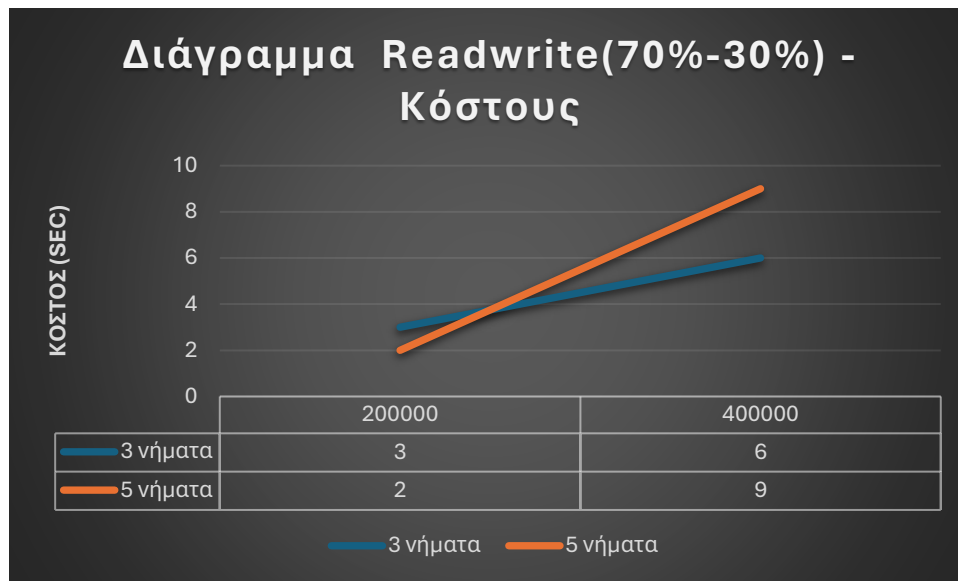
Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 8 δευτερόλεπτα και ότι χρειάζεται 20 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 91).

Κόστος readwrite τετρακοσίων χιλιάδων κλειδιών με ποσοστό εγγραφών 30% (120.000) και ποσοστό αναγνώσεων 70% (280.000) και με 5 νήματα ανάγνωσης:

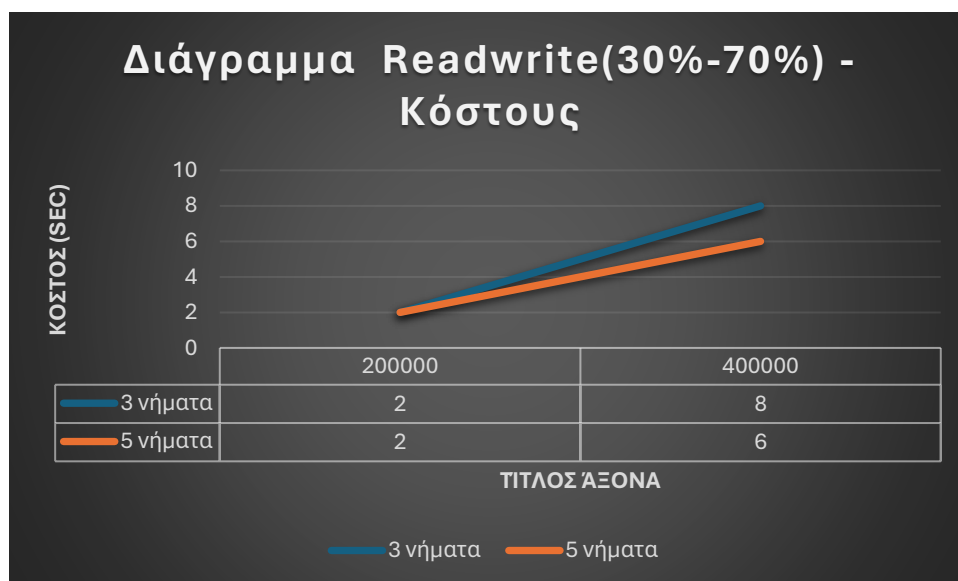
```
|Random-Read_Write      (done:400000): 0.000015 sec/op; cost:6.000(sec)|
```

Εικόνα 92

Τα αποτελέσματα που μας επιστράφηκαν ήταν: το κόστος ίσο με 6 δευτερόλεπτα και ότι χρειάζεται 15 μsec για να κάνει μια ανάγνωση ή μια εγγραφή κατά μέσω όρο (εικόνα 92).



Παρατηρούμε από το διάγραμμα ότι για μικρό πλήθος τιμών (60.000) για την λειτουργία του read τα 5 νήματα καταφέρνουν καλύτερη απόδοση. Όσο όμως το πλήθος των αναγνώσεων αυξάνεται τα 5 νήματα καταφέρνουν καλύτερη απόδοση.



Παρατηρούμε από το διάγραμμα ότι για μικρό πλήθος τιμών (140.000) για την λειτουργία του read οι δύο υλοποιήσεις καταφέρνουν την ίδια απόδοση. Όσο όμως το πλήθος των αναγνώσεων αυξάνεται τα 5 νήματα καταφέρνουν καλύτερη απόδοση.

Έξοδος της τελικής εντολής make

```
myy601@myy601lab1:~/kiwi/kiwi-source$ make all
cd engine && make all
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/engine'
CC db.o
CC memtable.o
CC indexer.o
CC sst.o
CC sst_builder.o
CC sst_loader.o
CC sst_block_builder.o
CC hash.o
CC bloom_builder.o
CC merger.o
CC compaction.o
CC skiplist.o
CC buffer.o
CC arena.o
CC utils.o
CC crc32.o
CC file.o
CC heap.o
CC vector.o
CC log.o
CC lru.o
AR libindexer.a
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/engine'
cd bench && make all
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/bench'
gcc -g -ggdb -Wall -Wno-implicit-function-declaration -Wno-unused-but-set-variable bench.c kiwi.c -L ../engine -lindexer -lpthread -lsnappy -o kiwi-bench
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/bench'
```

Πληροφορίες Συστήματος στο οποίο υλοποιήθηκε η άσκηση

```
CPU:          2 * Intel(R) Core(TM) i5-10600K CPU @ 4.10GHz
CPUCache:     12288 KB
```